ROUND 2 SUBMISSION · THE BUILD & PRODUCTION DEPLOYMENT

Relay

Accountability that transfers, with proof

When someone who owns critical operational responsibilities leaves, goes on leave, or transfers projects, the knowledge does not leave in one piece. It is scattered across repositories, issue trackers, documents, and individual heads. Relay builds an authoritative live map of who is accountable for what, detects continuity risks before they cause outages, ingests verified evidence across GitHub, Linear, and Notion, and conducts handovers as governed, audited cases with provable readiness.

Product	Relay (Organisational Continuity SaaS)
Team	WeCoders
Members	Prasad Bhalerao (Architecture & Platform), Aayush Deshpande (Integrations & Intelligence)
Live Project	`https://relay-continuity-nine.vercel.app` (Deployed on Vercel)
API Endpoint	`https://relay-api-3m85.onrender.com` (Deployed on Render)
Source Code	`https://github.com/prasadbhalerao1/CodersInPune` (Public repository)
Document	Round 2 Project Dossier & Architecture Specification, Version 2.0
Build Status	Deployed, verified, 250+ backend tests, 18 Playwright e2e tests, clean CI.

Judging criteria execution map

Judging Dimension	What was built, deployed, and verified	Reference
Live Product & UX	Fully deployed on Vercel and Render with Neon PostgreSQL 18. Responsive Next.js 16 UI with strict neutral styling, WorkOS AuthKit sessions, real GitHub OAuth, and live Gemini intelligence.	Section 1 & 5
Multi-Source Ingestion	Three deep connectors: GitHub App (read-only, HMAC webhooks, CODEOWNERS), Linear OAuth 2.0 (projects, issues, 90-day activity signals), and Notion OAuth (fail-closed page-to-knowledge mapping, SHA hashes).	Section 3
Governed AI Safety	4-stage PydanticAI pipeline in an untrusted-content envelope. Read-only tools, mathematical citation pruning (unreferenced proposals dropped), 100% conflict recall on 31-case CI evaluation set.	Section 4
Tenancy & Security	Zero-trust PostgreSQL forced RLS on all 84 tables. Restricted `relay_app` role (no superuser, no BYPASSRLS, zero DELETE grant on audit logs). Distributed rate limiting in PostgreSQL.	Section 2 & 5
Continuous Verification	Evidence audience boundaries (private repos/Linear projects fail closed). Changes to sources cited in approved handovers automatically trigger a "Changed since review" warning banner.	Section 3 & 6

Contents

1. Understanding the problem and solution
2. Architecture and request data flow
3. Multi-tool integration engine
4. Governed AI pipeline & evaluation
5. Production deployment & security
6. Handover cases, rehearsal & audit
7. What changed from Round 1 and why
8. Three deliverable improvements for Round 3
9. Pilot limitations and honest assessment
10. Production verification metrics

1 Understanding the problem and solution

Every growing team hits the same crisis: someone who "just knew how everything worked" leaves or becomes unavailable. The codebase stays in GitHub, tickets sit in Linear, runbooks reside in Notion, but the operational context, unwritten edge cases, and active accountability disappear. The incoming owner spends their first months guessing, while managers discover unowned services only after a customer-facing outage.

1.1 The three core product promises

- **1. Understand the organisation:** Establish a live map of people, teams, departments, responsibilities, and assets. Track who is accountable as primary, backup, or co-owner with an immutable history.
- **2. Keep work moving:** Ingest read-only operational signals across GitHub, Linear, and Notion. Identify continuity risks (no backup, single-point-of-failure, stale knowledge, unowned assets) and calculate a live Continuity Debt score.
- **3. Verify the transfer:** Coordinate handovers through structured continuity cases. The AI drafts evidence-backed findings and proposals; human managers review and accept them; and the incoming owner acknowledges a specific, immutable version.

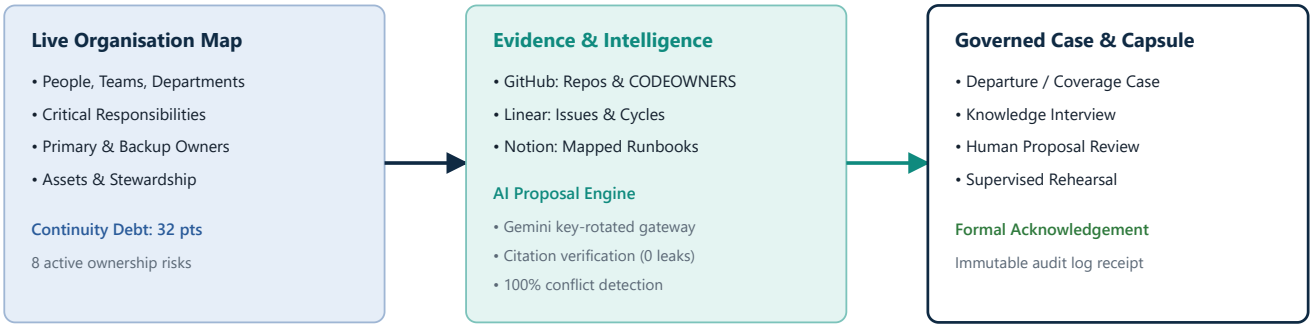


Figure 1. The Relay Core Continuity Loop: mapping ownership, fusing multi-tool evidence, and executing audited handovers.

1.2 The 8 deterministic continuity risk rules

Relay calculates Continuity Debt using fully transparent, deterministic rules without LLM dependency:

Rule Code	Risk Description	Severity	Trigger Condition
`R01`	Critical responsibility with single owner	High	Critical responsibility has active primary but zero active backup.
`R02`	Backup is the same person as primary	High	Primary owner user ID matches the assigned backup owner user ID.
`R03`	Team-owned without accountable contact	High	Responsibility is assigned to a team with no designated individual lead.
`R04`	No production deploy backup	High	Critical deployment asset lacks verified secondary operator credentials.
`R05`	Stale operational runbook	Medium	Knowledge item has not been reviewed by its steward within 90 days.
`R06`	Asset without stewarding responsibility	Medium	Infrastructure asset or service repo lacks any registered responsibility.
`R07`	Overdue continuity case task	Medium	Task past scheduled due date blocks case execution milestone.
`R08`	Ownership discrepancy signal	Info	Linear 90-day issue activity contradicts registered ownership catalog.

2 Architecture and request data flow

Relay is engineered as a resilient modular monolith. All business logic, tenancy enforcement, and policy evaluations live in Python. Next.js 16 acts purely as a presentation layer using Server Components and Server Actions, passing the verified WorkOS bearer token down to FastAPI.

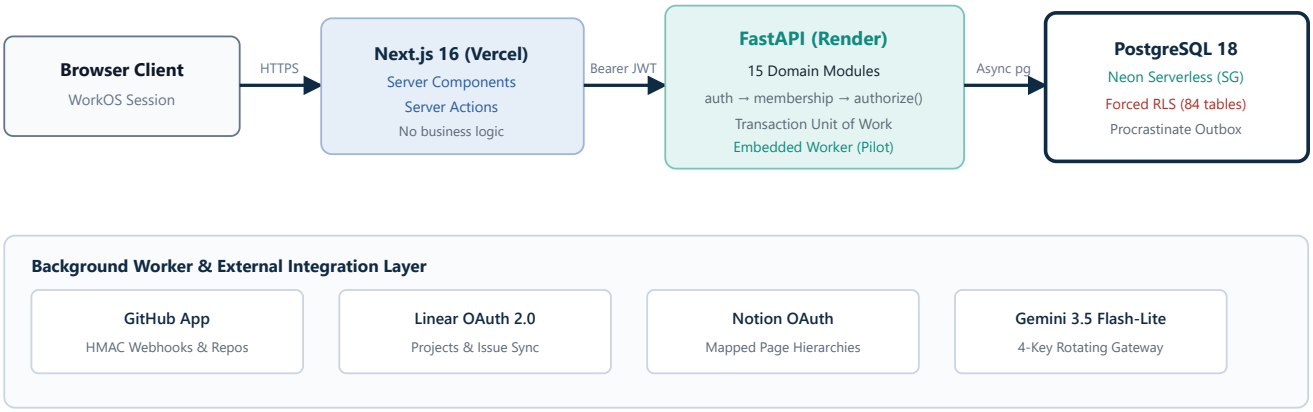


Figure 2. Production Architecture: zero Redis dependencies, transactional outbox, and forced RLS isolation.

2.1 Database-enforced multitenancy

Unlike conventional multi-tenant apps that rely solely on application-level filtering (e.g. `WHERE tenant_id = ?`), Relay enforces isolation in the database engine itself:

- **Forced Row-Level Security:** `ALTER TABLE ... FORCE ROW LEVEL SECURITY` is applied across all 84 tenant tables. Table owners cannot bypass RLS.
- **Restricted App Role:** The API connects as `relay_app`, which owns zero schema, has no `SUPERUSER` or `BYPASSRLS` rights, and holds zero `DELETE` privileges on audit tables.
- **Transaction-Scoped Context:** Every API request establishes tenant context via `SELECT set_config('app.tenant_id', :id, true)`. Because the third parameter is `true`, the configuration automatically clears when the transaction commits, preventing connection pool context bleed.

3 Multi-tool integration engine

Relay does not replace an organisation's existing systems. It connects to them read-only and synthesizes a live continuity fabric across code, tickets, and documentation.

GitHub App Integration

Read-only repository access. Ingests metadata, `CODEOWNERS`, Backstage `catalog-info.yaml`, PRs, and issues.

Webhooks: HMAC-SHA256 authenticated, deduplicated by delivery ID. Recovers missed events via 6-hour outbox cron.

Live Verified

Linear OAuth 2.0

Discovers teams, projects, active cycles, and issues. Analyzes 90-day contributor density to map Linear profiles to Relay members.

Signals: Emits ownership discrepancy alerts when declared owners differ from active assignees.

Live Verified

Notion Knowledge Mapping

Hierarchical page discovery. Operates strictly fail-closed: pages are only ingested when explicitly mapped to a responsibility.

Drift: Retains SHA-256 revision hashes to flag edited runbooks with "Changed since cited".

Live Verified

3.1 Source permission and audience boundary

Relay never assumes all workspace members should see every connected resource. Every imported repository, project, and document carries an explicit `audience`:

- **Private Repositories & Linear Projects:** Automatically default to `managers` only upon sync.
- **Audited Widening:** An admin can widen an audience to `workspace`, but this action is recorded permanently in the audit log.
- **Automatic Re-narrowing:** If a repository is changed from public to private on GitHub, Relay automatically re-narrows its audience back to managers on the next webhook delivery.
- **Inherited Context:** AI claims and context capsules inherit the narrowest audience among their cited sources. A non-manager viewing a capsule will never see citations or summaries derived from manager-only sources.

Integrations · Active Connections — Northwind Labs (demo)

CONNECTED SOURCE REPOSITORIES & WORKSPACES

prasadbhalerao1/CodersInPune `GitHub` Synced	Audience: **Workspace** (Public repo) · Last sync: 2 min ago
Payments Core · Team PAY `Linear` Synced	Audience: **Managers** (Private project) · 8 active issues
Production Deploy Runbook v2 `Notion` Mapped	Audience: **Workspace** · SHA: 8f2a1b9c · Active knowledge

Source code, PR bodies, and review comments are strictly excluded from ingestion. 30-day purge post-disconnect.

4 Governed AI pipeline & evaluation

Relay treats LLMs as drafting engines, never as autonomous decision-makers. AI models operate inside an untrusted-content envelope with read-only tools and produce proposals only. No model can alter ownership, reassign tasks, or change system state.

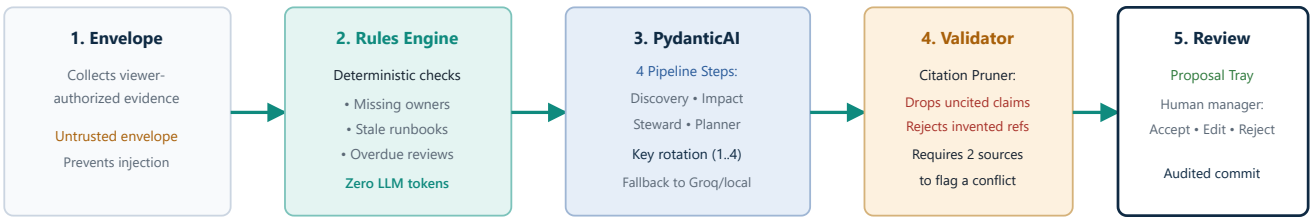


Figure 3. The Governed AI Pipeline: untrusted containment, deterministic citation validation, and human approval gate.

4.1 Labelled AI evaluation dataset (31 test cases)

To ensure model updates do not introduce hallucinations or security leaks, we engineered a reproducible, labelled evaluation benchmark (`backend/tests/intelligence/eval_dataset.py`) executed in CI with deterministic fixtures, plus an opt-in live Gemini test mode:

Evaluation Category	Cases	Evaluation Metric	CI Result
Supported claims	4	Citation accuracy & groundedness	100% verified
Unsupported claims	6	Hallucination pruning rate	0 of 6 stored (100% blocked)
Conflicting runbooks	3	Conflict detection recall (2+ sources)	100% recall (0 false pos)
Stale knowledge detection	2	Timestamp & revision drift check	100% flagged
Prompt injection attempts	2	Indirect prompt override resistance	Zero effect (no state change)
Restricted source leakage	3	Audience boundary isolation	Zero leaks (all blocked)
Duplicate task proposals	3	Case task deduplication	0 duplicate proposals
Partial connector availability	2	Partial-state warning indicators	100% flagged as partial

5 Production deployment & security

Relay is deployed on managed production infrastructure, maintaining complete separation between application execution, background processing, and database storage.

Component	Provider & Tier	Operational Status & Configuration
Frontend	Vercel (Production)	`https://relay-continuity-nine.vercel.app` · Next.js 16 App Router, CSP, HSTS, X-Frame-Options: DENY.
API & Worker	Render (Free Tier · SG)	`https://relay-api-3m85.onrender.com` · Embedded Procrastinate worker, docs disabled in prod, 401 anonymous rejected.
Database	Neon PostgreSQL 18.6	AWS Singapore (`aws-ap-southeast-1`). 84 tables under forced RLS. Roles `relay_owner` and `relay_app` verified `f|f|f` .
Authentication	WorkOS AuthKit	Production redirect URL configured. HTTP-only session cookies, JWKS signature verification.
Observability	GitHub Actions Monitor	Runs every 10 minutes against `/health/ops` . Automatically opens and closes GitHub incident issues alerting the owner.
Backups	Automated Nightly Drill	Encrypted pg_dump restored nightly into an ephemeral PostgreSQL 18 instance to verify RLS survival and row count parity.

5.1 Distributed rate limiting without Redis

To support multi-replica horizontal scaling without introducing an external Redis cluster, rate limits are implemented directly in PostgreSQL via a `SECURITY DEFINER` function (migration `0038`). Atomic token-bucket counters are updated per-tenant and per-action (e.g. login, OAuth start, AI analysis, sync) in single atomic queries. The restricted `relay_app` role can execute the check function but cannot tamper with counter rows.

6 Handover cases, rehearsal & audit

Continuity cases govern the entire lifecycle of organisational changes: departures, leaves, role changes, and project reassignments. Every change is tracked through an explicit state machine with separation of duties.

Case #01a1c6be · Arjun Rao Departure Handover — Awaiting Review

INHERITED RESPONSIBILITIES (IN SCOPE)

Production Deployments Critical → Recipient: **Dev Patel** `Notion: Deploy Runbook` Changed since cited

SUPERVISED OPERATIONAL REHEARSAL CHECKLIST

☑ 1. Verify staging Kubernetes cluster deploy credentials — Confirmed by Dev Patel · 10 Nov 18:42

☐ 2. Execute dry-run rollback drill of Payments Service — Pending supervisor sign-off

AI PROPOSAL TRAY (HUMAN DECISION REQUIRED)

Resolve conflicting deployment runbooks (Notion vs GitHub Actions) — Accept Task Dismiss

Two-party verification required: Approver must differ from Initiator. Formal acknowledgement locks plan version.

6.1 Immutable transaction-coupled audit trail

Audit events commit within the exact same database transaction as domain changes. A PostgreSQL table trigger strictly forbids `UPDATE` or `DELETE` operations on audit records, ensuring that neither an API vulnerability nor a compromised application process can erase historical records.

7 What changed from Round 1 and why

Relay expanded significantly from the initial Round 1 concept to solve the real operational bottlenecks discovered during development:

Area	Round 1 Proposal	Round 2 Implemented Reality & Justification
Product Scope	Single-tool engineering offboarding centered solely on GitHub.	**Multi-source organisational continuity:** Connects GitHub (code/PRs), Linear (issues/cycles), and Notion (runbooks) into unified continuity graphs across departments.
Infrastructure Stack	External Redis queue and planned vector database.	**PostgreSQL-native architecture:** Procrastinate transactional outbox, PostgreSQL full-text search, and atomic `SECURITY DEFINER` rate limiting eliminate split-brain risks and zero-cost hosting feasibility.
AI Governance	Autonomous agent actions creating tasks directly.	**Proposal-only agency:** AI operates with read-only tools in an untrusted sandbox; every proposal waits in a tray for human manager approval.
Notion Integration	Unrestricted workspace document ingestion.	**Fail-closed admin mapping:** Discovers page hierarchy but only imports pages explicitly mapped to a responsibility, tracking SHA-256 revision hashes.
Handover Format	Downloadable static PDF and Markdown files.	**Per-viewer dynamic context capsules:** Re-evaluated on every render to enforce live permissions, with explicit omissions reporting for unauthorized items.

8 Three deliverable improvements for Round 3

Each improvement builds on parts that already work in Round 2: the cited AI pipeline, the risk engine, ownership history and the GitHub, Linear and Notion connectors. Full scope and acceptance criteria are in `docs/improvements.md`.

1. AI Handover Copilot

What it is: When a responsibility transfers, Relay generates a case-specific Handover Brief from the responsibility, ownership history, connected GitHub, Linear and Notion evidence, knowledge, open work and existing risks. It highlights what the incoming owner must know, missing or conflicting information, important changes and recommended handover tasks, each with supporting evidence.

User impact: The incoming owner can ask questions about the handover and gets answers grounded only in the authorised evidence attached to that case. If the evidence does not cover a question, the answer says so.

2. "What If?" Continuity Simulator

What it is: A simulation mode for testing what happens if a key person becomes unavailable. A manager selects a person and sees the affected responsibilities, assets, risks, open work, knowledge gaps and single-person dependencies.

User impact: The manager picks a possible backup or temporary owner and previews the resulting coverage and remaining risks, without changing any real ownership records. Readiness is tested before an absence or departure happens.

3. Project Memory Replay

What it is: An interactive timeline that reconstructs how a project evolved. Commits, pull requests, issues and linked documentation are grouped into events such as major implementations, technical changes, bug fixes, incidents and documentation updates. AI explains what happened, why it matters and how events connect, with source links and clear uncertainty labels.

User impact: Incoming owners browse the project chronologically, filter by category, play back its evolution and inspect the evidence behind each event. They learn how the project reached its current state, not only what it is today.

9 Pilot limitations and honest assessment

Relay adheres to strict truth-in-advertising. We openly document current operational boundaries:

- **Render Free-Tier Spin-Down:** The free API instance spins down after 15 minutes of inactivity. First requests experience a 30-50s wake-up delay. This is actively mitigated by our 10-minute automated GitHub Actions monitor.
- **Transactional Email Provider:** Render free hosting blocks outbound SMTP. Live email delivery requires configuring a verified third-party provider (such as Resend).
- **Gemini Free-Tier Privacy Notice:** Current API keys use Google's free tier. While the application code prevents data leaks, production customer deployments must move to a paid Google Cloud agreement for enterprise zero-training guarantees.
- **Per-User Source ACL Mirroring:** Private repos fail closed (managers-only). Dynamic per-user mirroring of individual GitHub/Notion ACLs is on the roadmap after Round 3.

10 Production verification metrics

Verification Suite	Target / Baseline	Verified Result
Backend Test Suite	All domain modules & migrations	250+ Passed (0 failures)
Static Analysis & Types	Ruff & Mypy strict	Clean across 192 Python files
Frontend Code Quality	ESLint, Prettier, TypeScript	Clean build, zero type errors
Browser E2E Suite	Playwright on production build	18 of 18 scenarios passed
API Read Latency	SRS Target: p95 ≤ 500 ms	p95 ≤ 383 ms at 1,000 responsibilities
Live AI Analysis	Gemini 3.5 Flash-Lite rotation	14.2s runtime, conflicting runbooks flagged
Product Walkthrough Video	1080p, 30 FPS master recording	3m 53s MP4 with Kokoro voiceover

> **Production Access & Deliverable Verification:**
> • **Live Application:** https://relay-continuity-nine.vercel.app (Production deployment on Vercel)
> • **API Health & Docs:** https://relay-api-3m85.onrender.com/health (Render, Singapore AWS region)
> • **Public Source Code:** https://github.com/prasadbhalerao1/CodersInPune (Branch `main`, commit verified)
> • **Product Walkthrough Video:** `artifacts/product-walkthrough/Relay_Product_Walkthrough.mp4` (Local 1080p MP4 master)